

Métodos y descomposición

Cómo dividir un programa en piezas pequeñas, claras y reutilizables

Idea central

Un método agrupa instrucciones para resolver una tarea concreta.

Ventaja

El código se entiende mejor y se evita repetir lo mismo.

Meta

Que el programa principal coordine y los métodos trabajen.

Qué vamos a aprender

Resultados del módulo

Crear métodos con parámetros y valor de retorno.
Diferenciar variables locales y ámbito.
Sobrecargar métodos de forma sencilla.
Dividir un problema grande en partes más pequeñas.

Plan recomendado

Explicación breve → ejemplo guiado → práctica individual → corrección.

Clave docente

No basta con que funcione: el alumnado debe poder explicar qué hace cada método.

Por qué usamos métodos

Un programa largo se vuelve difícil de leer, probar y corregir

```
// Sin métodos: todo mezclado  
leer datos  
calcular total  
validar precio  
mostrar resultado
```



```
// Con métodos  
leerDatos();  
calcularTotal();  
validarPrecio();  
mostrarResultado();
```

Beneficio

Cada parte tiene una responsabilidad clara.

Qué es un método

Un bloque de código con nombre que podemos llamar cuando lo necesitamos

```
public static void saludar() {  
    System.out.println("Hola");  
}  
  
// Llamada al método  
saludar();
```

Tiene un nombre: saludar.
Se ejecuta cuando lo llamamos.
Puede recibir datos.
Puede devolver un resultado.

Partes de un método

La cabecera indica cómo se comporta; el cuerpo indica qué hace

```
public static double calcularTotal(double precio, int cantidad) {  
    return precio * cantidad;  
}
```

public static

Permite llamar al método desde main en estos primeros ejercicios.

double

Tipo de dato que devuelve el método.

parámetros

Datos que recibe para poder trabajar.

return

Devuelve el resultado calculado.

Métodos void

Se usan cuando el método realiza una acción, pero no devuelve un dato

```
public static void mostrarTitulo() {  
    System.out.println("Factura");  
}  
  
public static void main(String[] args) {  
    mostrarTitulo();  
}
```

void

Significa: este método no devuelve ningún valor.

Uso típico

Mostrar mensajes, imprimir menús, enseñar resultados.

Métodos con return

Se usan cuando necesitamos que el método calcule y entregue un resultado

```
public static int sumar(int a, int b) {  
    return a + b;  
}
```

```
int resultado = sumar(4, 7);  
System.out.println(resultado);
```

El método recibe dos números.
Calcula la suma.
Devuelve el resultado con return.
Ese resultado se guarda en una variable.

Parámetros

Son variables que el método recibe desde fuera

```
public static double calcularTotal(double precio, int
cantidad) {
    return precio * cantidad;
}

double total = calcularTotal(12.50, 4);
```

precio = 12.50

cantidad = 4

total = 50.0

Importante

El orden de los argumentos debe coincidir con el orden de los parámetros.

Variables locales

Una variable creada dentro de un método solo existe dentro de ese método

```
public static void ejemplo() {  
    int numero = 10;  
    System.out.println(numero);  
}  
  
public static void main(String[] args) {  
    ejemplo();  
    // System.out.println(numero); // error  
}
```

Ámbito

El ámbito es la zona donde una variable puede utilizarse.

Error típico

Intentar usar fuera una variable que nació dentro del método.

main como coordinador

El método main no debería hacerlo todo

```
public static void main(String[] args) {  
    double precio = 12.50;  
    int cantidad = 4;  
  
    double total = calcularTotal(precio, cantidad);  
    mostrarResultado(total);  
}
```

main prepara los datos.
main llama a otros métodos.
Cada método resuelve una parte.
El programa queda más limpio.

Descomponer un problema

Antes de escribir código, dividimos la tarea en pasos



```
calcularSubtotal(precio, cantidad)
calcularIVA(subtotal)
mostrarFactura(subtotal, iva, total)
```

Ejemplo completo: calculadora

Método sencillo con parámetros y retorno

```
public class Calculadora {  
  
    public static int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        int resultado = sumar(4, 7);  
        System.out.println(resultado);  
    }  
}
```

Qué practica

Crear un método, enviarle datos y recoger el resultado.

Resultado

El programa muestra 11.

Ejemplo: calcular total

Reutilizamos el ejercicio de precio y cantidad

```
public static double calcularTotal(double precio, int cantidad) {  
    return precio * cantidad;  
}  
  
public static void main(String[] args) {  
    double precio = 12.50;  
    int cantidad = 4;  
    double total = calcularTotal(precio, cantidad);  
    System.out.println("Total: " + total);  
}
```

El cálculo queda separado.

Si cambia la fórmula, solo se toca un método.

El código principal se entiende mejor.

Ejemplo: áreas geométricas

Tres métodos, tres responsabilidades

```
static double areaRectangulo(double base, double altura) {  
    return base * altura;  
}  
  
static double areaCirculo(double radio) {  
    return Math.PI * radio * radio;  
}  
  
static double areaTriangulo(double base, double altura) {  
    return base * altura / 2;  
}
```

Buena práctica

Cada método calcula solo una cosa.

Nombre claro

areaRectangulo explica exactamente qué hace.

Nombres de métodos

Deben expresar una acción clara

Buenos nombres

calcularTotal
validarEdad
mostrarMenu
leerPrecio

Malos nombres

hacer
cosas
metodo1
calculo

Regla práctica

Verbo + dato sobre el que actúa.
Ejemplo: calcularMedia,
mostrarResultado.

Un buen nombre reduce la necesidad de explicar el código.

Un método, una tarea

Evita métodos enormes que mezclan lectura, cálculo y presentación

```
// Mejor separar:  
leerDatos();  
calcularResultado();  
mostrarResultado();
```

Lectura

Pedir o preparar datos

Cálculo

Procesar información

Salida

Mostrar resultado

Objetivo

Que cada método se pueda probar y corregir de forma aislada.

Sobrecarga de métodos

Varios métodos pueden llamarse igual si reciben parámetros distintos

```
public static int sumar(int a, int b) {  
    return a + b;  
}  
  
public static double sumar(double a, double b) {  
    return a + b;  
}
```

Mismo nombre: sumar.

Distinto tipo de parámetros.

Java elige cuál usar según los datos enviados.

Útil, pero no conviene abusar al principio.

Validación con métodos

Podemos crear métodos que devuelvan true o false

```
public static boolean esMayorEdad(int edad) {  
    return edad >= 18;  
}  
  
public static void main(String[] args) {  
    boolean resultado = esMayorEdad(20);  
    System.out.println(resultado);  
}
```

boolean

Un método también puede devolver verdadero o falso.

Uso

Muy útil para validar datos antes de continuar.

Validar contraseña

Ejercicio guiado del módulo

```
public static boolean esPasswordValida(String password) {  
    return password.length() >= 8;  
}  
  
public static void main(String[] args) {  
    System.out.println(esPasswordValida("Java2026"));  
}
```

Primera versión: comprobar longitud.
Después añadiremos mayúsculas.
Después añadiremos dígitos.
Se puede mejorar por partes.

Descomposición progresiva

No se resuelve todo a la vez: se construye paso a paso.

Práctica guiada

Convertir un ejercicio anterior en métodos

```
public static double calcularMedia(double n1, double n2,
double n3) {
    return (n1 + n2 + n3) / 3;
}

public static void mostrarMedia(double media) {
    System.out.println("Media: " + media);
}
```

Antes

Todo el cálculo estaba en main.

Después

El cálculo vive en calcularMedia().

Ejercicio 1

Calcular áreas con métodos

Crea una clase llamada BibliotecaMatematica.
Crea tres métodos: areaRectangulo, areaCirculo y areaTriangulo.
Llama a los métodos desde main.
Muestra cada resultado por pantalla.

```
// Pista
static double areaRectangulo(double
base, double altura) {
    return base * altura;
}
```

Ejercicio 2

Factura descompuesta

Crea métodos para calcular subtotal, IVA y total.

Usa precio = 12.50 y cantidad = 4.

El IVA será del 21 %.

Muestra todos los resultados desde main.

```
calcularSubtotal(precio, cantidad)
calcularIVA(subtotal)
calcularTotal(subtotal, iva)
```

Ejercicio 3

Validación sencilla

Crea un método `esNotaValida(double nota)`.
Debe devolver `true` si la nota está entre 0 y 10.
Debe devolver `false` si está fuera de rango.
Prueba el método con varias notas.

```
public static boolean  
esNotaValida(double nota) {  
    return nota >= 0 && nota <= 10;  
}
```

Cierre del módulo

Lo importante antes de pasar a arrays y colecciones

Sé crear un método con parámetros.

Sé devolver un resultado con return.

Entiendo la diferencia entre void y un tipo de retorno.

Sé dividir un problema en métodos pequeños.

Puedo explicar qué hace cada método.

Idea clave

Un método no es solo código: es una pequeña responsabilidad con nombre.



Resumen final

Métodos = orden, claridad y reutilización

1

Nombrar bien

2

Separar tareas

3

Usar parámetros

4

Devolver resultados

```
public static double calcularTotal(double precio, int
cantidad) {
    return precio * cantidad;
}
```